

RELATÓRIO TÉCNICO DE MIGRAÇÃO DE BANCO RELACIONAL PARA NOSQL ORIENTADO A DOCUMENTOS

ESTUDO DE CASO: LOJA VIRTUAL DE LEAGUE OF LEGENDS — DO POSTGRESQL AO MONGODB ATLAS

Curso: Pós-Graduação em Data Science e Analytics

Unidade Curricular: Banco de Dados e Big Data para Data Science

Unidades de Aprendizagem: UA 01 e UA 02

Equipe de Desenvolvimento:

- ****Diogo Galvão Carvalho****
 - ****Felipe Artur Macedo Lima****
 - ****Luan Cavalcante Dias Rodrigues****

Repositório do Projeto: https://github.com/FelipeArtur/bigdata_datascience

Data: Setembro de 2026

1. INTRODUÇÃO

Nas últimas duas décadas, a transição para sistemas de computação em escala de internet e a digitalização massiva de serviços introduziram desafios sem precedentes na gestão e persistência de dados. O paradigma relacional tradicional fundamentado na Álgebra Relacional de Edgar F. Codd e nas propriedades ACID (Atomicidade, Consistência, Isolamento e Durabilidade), embora insubstituível para transações financeiras estritas e esquemas altamente previsíveis, apresenta gargalos de desempenho e escalabilidade quando submetido a cargas de trabalho analíticas ou transacionais de altíssima concorrência com esquemas em rápida evolução.

Dentre os principais atritos do modelo relacional em ambientes modernos, destacam-se:

1. **Sobrecarga de Junções (JOIN s):** A busca por normalização estrita (da 1ª à 3ª Forma Normal) fragmenta os dados de uma mesma entidade de negócio em dezenas de tabelas correlacionadas. Em cenários de leitura intensiva (*read-heavy*), a reconstrução da visão completa exige múltiplas junções computacionalmente onerosas, multiplicando operações de I/O em disco e consumo de memória;

2. **Impedância Objeto-Relacional:** A disparidade conceitual entre o modelo tabular bidimensional relacional e as estruturas orientadas a objetos utilizadas no desenvolvimento de software moderno obriga a utilização de complexas camadas intermediárias de mapeamento (ORMs), adicionando latência e complexidade arquitetural;

3. **Dificuldade de Escalabilidade Horizontal:** Os RDBMS foram historicamente desenhados para escala vertical (*scale-up* — aumento de processadores e memória em um único servidor). Particionar horizontalmente (*sharding*) um banco relacional com integridade referencial distribuída e transações multi-tabela é uma tarefa de alta fricção operacional.

Como resposta a essas demandas, consolidou-se o movimento NoSQL (*Not Only SQL*), destacando-se os bancos de dados orientados a documentos, cujo principal expoente é o **MongoDB**. Ao estruturar os dados em documentos BSON (*Binary JSON*) semiestruturados e hierárquicos, o MongoDB permite agregar informações correlacionadas em estruturas autocontidas, viabilizando operações de leitura sem junções, particionamento horizontal nativo e evolução flexível de atributos.

Objetivo deste Trabalho:

Este projeto tem como meta projetar, implementar e validar tecnicamente a migração de um sistema transacional de comércio eletrônico no segmento gamer — a **Loja Virtual de League of Legends (LoL)**. O sistema, originalmente modelado em banco relacional (**PostgreSQL**) composto por 7 tabelas e mais de 1.000 registros transacionais, foi migrado para o banco NoSQL orientado a documentos (**MongoDB Atlas**). O relatório detalha a arquitetura relacional de origem, fundamenta as decisões de modelagem (*embedding* versus *referencing*), avalia os aspectos de escalabilidade e elasticidade, demonstra o código de migração e validações no Atlas, e conclui com uma avaliação crítica dos impactos e trade-offs técnicos obtidos.

2. DESCRIÇÃO DO CENÁRIO E MODELO RELACIONAL DE ORIGEM

2.1 Contextualização do Domínio de Negócio

O cenário selecionado modela a economia transacional do jogo eletrônico *League of Legends* (Riot Games), um dos títulos mais jogados do mundo na modalidade MOBA (*Multiplayer Online Battle Arena*). Durante as partidas competitivas (disputadas entre duas equipes de 5 jogadores cada), os competidores acumulam recursos (ouro e pontos de experiência) e compram itens virtuais em tempo real para potencializar os atributos de seus campeões.

Cada transação na loja envolve uma rede de relações interdependentes: o jogador adquirente, sua qualificação competitiva (*elo*), a região geográfica do servidor, as características do item adquirido (custo, atributos, categorias de efeito), e a partida específica onde a aquisição se concretizou.

2.2 Estrutura Relacional (7 Tabelas Normalizadas)

Para superar os requisitos mínimos estipulados na atividade (mínimo de 3 tabelas e 20 registros), o banco relacional foi projetado em 3ª Forma Normal com **7 tabelas estruturadas** e um volume total de **1.043 compras, 254 itens reais** extraídos da API oficial do jogo, **32 categorias, 30 partidas e 25 jogadores** distribuídos em ranques competitivos:

Tabela	Descrição	Chave Primária (PK)	Chaves Estrangeiras (FK)	Total de Linhas
elo	Tabela de domínio dos ranques competitivos (Ferro a Desafiante).	`id_elo`	—	10
categoria	Domínio de classificações funcionais de itens (Boots, ManaRegen, Damage, etc.).	`id_categoria`	—	32
item	Catálogo de itens disponíveis com atributos, custo em ouro e descrição técnica.	`item_id`	—	254
item_categoria	Tabela associativa que resolve o relacionamento Muitos-para-Muitos (N:N) entre itens e categorias.	`(item_id, id_categoria)`	`item_id → item` `id_categoria → categoria`	834
jogador	Cadastro de jogadores com nick, ranque e região competitiva (BR, KR, EUW).	`id_jogador`	`id_elo → elo`	25
partida	Registro das partidas realizadas, data, horário, duração e resultado (Vitória/Derrota).	`id_partida`	—	30
compra	Tabela fato transacional registrando as aquisições durante as partidas.	`id_compra`	`id_jogador → jogador` `id_item → item` `id_partida → partida`	1.043

2.3 Cardinalidade e Relacionamentos do Modelo Relacional

- **elo (1) — (N) jogador** Um elo agrupa múltiplos jogadores; cada jogador possui um único elo ativo.
 - **item (N) <—> (M) categoria** (via `item\_categoria`)** Um item pode pertencer a múltiplas categorias funcionais (ex: Botas com Resistência Mágica e Tenacidade); uma categoria engloba dezenas de itens distintos.
 - **jogador (1) — (N) compra** Um jogador realiza múltiplas compras ao longo de sua trajetória.
 - **item (1) — (N) compra** Um item do catálogo pode ser comercializado em inúmeras transações.
 - **partida (1) — (N) compra** Cada partida hospeda as compras dos 10 participantes envolvidos no confronto.

2.4 Script DDL e Consulta Relacional de Alto Custo (Multi-JOIN)

No modelo relacional, para obter o extrato detalhado de compras de um jogador com dados completos do item, ranque e partida, o motor SQL precisa avaliar 6 tabelas em cascata:

```
-- Consulta Relacional: Histórico consolidado do jogador com 6 tabelas unidas
SELECT
  j.nick,
  j.regiao,
  e.elo,
  p.data_partida,
```

```
p.resultado,  
i.nome AS item,  
c.preco_unitario,  
c.quantidade,  
c.total_compra,  
c.minuto_compra  
FROM compra c  
JOIN jogador j ON c.id_jogador = j.id_jogador  
JOIN elo e ON j.id_elo = e.id_elo  
JOIN item i ON c.id_item = i.item_id  
JOIN partida p ON c.id_partida = p.id_partida  
WHERE j.nick = 'Faker'  
ORDER BY c.data_compra, c.minuto_compra;
```

Em alta concorrência, a necessidade de ler índices distintos e executar operações de *Hash Join* ou *Nested Loops* sobre discos magnéticos ou SSDs gera gargalos de throughput e locks de concorrência.

3. MODELO NOSQL IDEALIZADO (MONGODB)

3.1 Filosofia de Modelagem Orientada a Documentos

Diferentemente dos RDBMS, onde a prioridade é a eliminação de redundâncias, no MongoDB o princípio fundamental de design é: **"dados que são acessados juntos devem ser armazenados juntos"** (*Data that is accessed together should be stored together*). A modelagem orientada a documentos busca alinhar o layout físico de armazenamento com os padrões reais de leitura e escrita da aplicação.

O modelo proposto reorganizou as 7 tabelas relacionais em **3 coleções estruturadas no MongoDB**:

```
[Banco Relacional: 7 Tabelas]
├── elo
├── categoria
├── item ──────────────────> Coleção 'itens' (Referencing + categorias embutidas)
├── item_categoria
├── jogador ───────────────> Coleção 'jogadores' (Embedding: elo + array de compras)
├── partida ───────────────> Coleção 'partidas' (Referencing)
└── compra
```

3.2 Decisões de Modelagem: Embedding versus Referencing

A) Embedding (Incorporação) Aplicado à Coleção `jogadores`:

- ****Elo Embutido:**** O elo competitivo é incorporado como um atributo textual direto ("elo": "Desafiante"). Não há ganho em manter uma coleção isolada para ranques, uma vez que a leitura do jogador quase invariavelmente requer a exibição de seu ranking e a lista de elos é estática.
- ****Histórico de Compras Embutido:**** O array `compras: [...]` é armazenado dentro do documento do respectivo jogador. Em mais de 90% das requisições de clientes em lojas virtuais (telas de "Meu Perfil", "Meu Inventário", "Histórico de Transações Recentes"), as compras são lidas no escopo daquele usuário. Embutir as compras elimina a necessidade de `JOIN`s com a tabela transacional, permitindo que o MongoDB retorne todo o inventário do jogador em um único *Index Seek* no campo `\_id`.
- ****Snapshot Pattern (Preço Histórico no Momento da Compra):**** Dentro do subdocumento de compra, embutimos o nome do item, preço unitário, quantidade, valor total e tags de categorias. Essa denormalização deliberada preserva a ****imutabilidade contábil****: se o catálogo de itens alterar o preço de uma espada de 350 para 400 de ouro no próximo balanceamento do jogo, as compras realizadas no passado continuarão refletindo fielmente os 350 de ouro pagos na data original.
- ****Atomicidade de Atualização:**** No MongoDB, operações em um único documento são estritamente atômicas. Adicionar uma nova compra ao jogador por meio do operador `\$push` e incrementar seu total de ouro gasto via `\$inc` ocorre em uma única instrução transacional, sem exigir locks globais em tabelas separadas.

B) Referencing (Referência) Aplicado às Coleções `itens` e `partidas`:

- ****Catálogo de `itens` como Coleção Referenciada:**** O catálogo contém centenas de itens consultados por toda a base de usuários para renderização da vitrine da loja, busca e balanceamento. Embutir a totalidade dos dados do catálogo dentro de cada compra provocaria uma inflação inaceitável de bytes e acarretaria anomalias de atualização se metadados globais (como ícone ou descrição funcional) precisassem ser ajustados. Mantemos a coleção `itens` independente, referenciando seu `item\_id` nas compras.
- ****Eliminação de Tabela N:N (`item\_categoria`):**** No catálogo `itens`, incorporamos diretamente um array de categorias ("categorias": ["Boots", "Speed"]). Isso extingue a necessidade de uma tabela intermediária N:N, reduzindo em 100% a complexidade relacional dessa dimensão.
- ****Histórico de `partidas` como Coleção Referenciada:**** Uma partida é uma entidade autônoma com métricas próprias (duração, data, horário, resultado global) que pertence simultaneamente a 10 jogadores. Incorporar os dados da partida repetidamente em cada compra de cada

um dos 10 competidores causaria duplicação descontrolada. As compras armazenam apenas a chave de referência `id\_partida`.

C) Análise do Limite de 16 MB por Documento BSON

O MongoDB possui uma restrição de tamanho máximo de 16 megabytes por documento individual BSON. Realizamos o dimensionamento volumétrico:

- Cada subdocumento de compra na estrutura proposta ocupa aproximadamente **180 bytes** codificado em BSON.
- Um jogador com 1.000 compras acumuladas consome cerca de **180 KB**, correspondendo a apenas **1,1%** do limite de 16 MB.
- Portanto, para o horizonte operacional da aplicação, o modelo de incorporação é seguro e extremamente eficiente. Caso um jogador atinja dezenas de milhares de compras em longo prazo, a arquitetura pode aplicar o padrão de *Bucketing* ou arquivamento de temporadas (*seasons*).

3.3 Escalabilidade e Elasticidade no MongoDB

O modelo NoSQL idealizado provê suporte nativo a dois pilares fundamentais da computação em nuvem:

1. Escalabilidade Horizontal (Sharding): Ao contrário do PostgreSQL, cuja partição exige extensões ou regras manuais complexas, o cluster MongoDB Atlas pode distribuir os documentos da coleção `jogadores` entre múltiplos shards. Utilizando a chave de shard (`regiao`, `_id`), as operações de escrita e leitura de jogadores da Coreia (KR) e do Brasil (BR) são roteadas diretamente aos nós correspondentes sem contenção de recursos.

2. Elasticidade e Schema Flexível: O schema flexível viabiliza a introdução de novos campos (como eventos sazonais, passes de batalha, cosméticos temáticos ou dados de telemetria de latência) sem downtime e sem requisições `ALTER TABLE` bloqueantes, assegurando agilidade no ciclo de desenvolvimento contínuo (CI/CD).

3.4 Exemplos Ilustrativos de Documentos BSON/JSON

Exemplo 1: Documento da Coleção `jogadores` (Embedding de Elo e Compras)

```
{
  "_id": 1,
  "id_jogador": 1,
  "nick": "Faker",
  "regiao": "KR",
  "elo": "Desafiante",
  "total_gasto_ouro": 80525,
  "total_itens_adquiridos": 41,
  "total_transacoes": 37,
  "compras": [
    {
      "id_compra": 72,
      "id_item": 3508,
      "item": "Colhedor de Essência",
      "categorias": ["Damage", "CriticalStrike",
      "preco_unitario": 3050,
      "quantidade": 1,
```

```
"total_ouro": 3050,  
"data_compra": "2025-08-21",  
"minuto_compra": 18,  
"id_partida": 3  
},  
{  
  "id_compra": 73,  
  "id_item": 1036,  
  "item": "Espada Longa",  
  "categorias": ["Damage", "Lane"],  
  "preco_unitario": 350,  
  "quantidade": 1,  
  "total_ouro": 350,  
  "data_compra": "2025-08-21",  
  "minuto_compra": 22,  
  "id_partida": 3  
}  
]  
}
```

Exemplo 2: Documento da Coleção `itens` (Referencing + Categorias Embutidas)

```
{  
  "_id": 1001,  
  "item_id": 1001,  
  "nome": "Botas",  
  "preco_unitario": 300,  
  "descricao": "25 de Velocidade de Movimento",  
  "categorias": ["Boots", "MovementSpeed"]  
}
```

Exemplo 3: Documento da Coleção `partidas` (Referencing)

```
{  
  "_id": 1,  
  "id_partida": 1,  
  "data_partida": "2025-08-15",  
  "hora_partida": "14:32:10",  
  "duracao_minutos": 36,  
  "resultado": "Derrota"  
}
```

4. IMPLEMENTAÇÃO DA MIGRAÇÃO E VALIDAÇÃO NO MONGODB ATLAS

4.1 Pipeline de Extração, Transformação e Carga (ETL)

A migração foi automatizada através de script em Python (`scripts/migrate_to_mongodb.py`), utilizando as bibliotecas padrão e o driver oficial `pymongo`. O fluxo operacional seguiu os passos:

1. Extração dos registros das tabelas normalizadas exportadas em arquivos CSV;

2. Construção de tabelas hash em memória para resolução ágil de chaves estrangeiras (`id_elo`, `id_categoria`);

3. Consolidação dos arrays de categorias nos itens e dos históricos de transações no array embutido de cada jogador;

4. Serialização dos documentos nos arquivos `jogadores.json`, `itens.json` e `partidas.json`;

5. Carga remota segura no cluster M0 do **MongoDB Atlas** utilizando autenticação via TLS e string de conexão com credenciais protegidas via variáveis de ambiente.

4.2 Validações e Consultas de Desempenho

Após a conclusão da ingestão, foram executadas baterias de consultas para comprovar a integridade dos dados e a expressividade da API do MongoDB:

Validação 1: Leitura de Perfil Completo com `findOne()`

```
# Consulta atômica sem JOIN para recuperar o in
faker = db.jogadores.find_one({"nick": "Faker"})
print(f"Jogador: {faker['nick']} | Elo: {faker[
print(f"Total de itens no inventário: {len(faker
```

Resultado: Retorno instantâneo em uma única leitura física de documento, trazendo todas as compras realizadas pelo jogador.

Validação 2: Filtro por Região com Projeção Seletiva

```
# Recuperação dos competidores da região KR
cursor_kr = db.jogadores.find({"regiao": "KR"},
for jog in cursor_kr:
    print(jog)
```

Resultado: O MongoDB utiliza o índice na coluna `regiao`, retornando apenas a projeção solicitada sem necessidade de deserializar o array de compras.

Validação 3: Pipeline de Agregação Analítica (`aggregate`)

```
# Ranking das regiões por receita acumulada na
pipeline = [
  {"$group": {
    "_id": "$regiao",
    "receita_total": {"$sum": "$total_gasto"},
    "jogadores_ativos": {"$sum": 1},
    "media_gasto": {"$avg": "$total_gasto_o
  }},
  {"$sort": {"receita_total": -1}}
]
resultados = list(db.jogadores.aggregate(pipeline))
```

Resultado: Processamento nativo do framework de agregação do MongoDB, permitindo análises sumárias rápidas diretamente no cluster.

5. CONCLUSÃO E AVALIAÇÃO CRÍTICA

A execução prática deste projeto de migração permitiu confrontar as características teóricas e práticas dos sistemas de gerenciamento de banco de dados relacionais e NoSQL, gerando aprendizados aprofundados sobre arquitetura de dados moderna.

5.1 Síntese dos Impactos e Benefícios Observados

1. **Desempenho de Leitura Otimizado:** No modelo relacional original, recuperar o histórico transacional do jogador requeria a junção de 6 tabelas com scans de múltiplos índices. No MongoDB Atlas, a mesma informação reside em um documento contíguo, reduzindo a latência de rede e a contenção de memória a níveis mínimos;

2. **Simplificação Estrutural:** A eliminação da tabela de relacionamento N:N `item_categoria` em favor de arrays embutidos demonstrou como o modelo orientado a documentos é mais intuitivo e conciso para tratar coleções de tags ou categorias finitas;

3. **Preservação de Integridade Contábil via Denormalização Controlada:** O uso deliberado do **Snapshot Pattern** protegeu o sistema de erros históricos decorrentes de alterações de preço no catálogo, comprovando que a desnormalização, quando planejada, é uma ferramenta técnica legítima e poderosa;

4. **Alinhamento com Arquitetura de Nuvem:** A separação limpa entre coleções com embedding (`jogadores`) e coleções com referencing (`itens`, `partidas`) viabilizou o particionamento horizontal (**sharding**) sem as barreiras impostas pelas chaves estrangeiras relacionais.

5.2 Limitações e Desafios Conceituais

1. **Ausência de Integridade Referencial Declarativa Estrita:** No MongoDB, a responsabilidade de garantir que um `id_partida` referenciado em uma compra realmente exista na coleção `partidas` é transferida para a camada de aplicação ou para validações de schema BSON (`jsonSchema`). Não há gatilhos (**foreign key constraints**) nativos automáticos com deleção em cascata;

2. **Custo de Atualizações Globais em Dados Denormalizados:** Se o nome de um item mudar por razões de direitos autorais ou tradução, o documento correspondente na coleção `itens` é atualizado instantaneamente, mas as compras já realizadas mantêm o nome prévio. Caso a regra de negócio exigisse a sincronização em cascata de todas as compras passadas, seria necessário um script de atualização em massa (`updateMany` com filtros de array), o que consumiria elevado tempo de computação.

5.3 Próximos Passos

Os dados estruturados e migrados para o MongoDB Atlas constituem a fundação exata para a segunda etapa do projeto (**UA 03 e UA 04**). Na etapa subsequente, os dados transacionais do MongoDB e os metadados demográficos complementares serão ingeridos no **Apache Spark** via **Databricks Community Edition**, onde serão aplicadas otimizações de **Broadcast Join**, **Caching** em memória e gravação analítica no formato **Delta Lake**.

6. REFERÊNCIAS BIBLIOGRÁFICAS

- BARR, M.; LIOR, G.; MOLLY, V. ****Fundamentos da qualidade de dados: guia prático para criar pipelines de dados confiáveis****. Rio de Janeiro: Alta Books, 2024.
- BOAGLIO, F. ****MongoDB: construa novas aplicações com novas tecnologias****. São Paulo: Casa do Código, 2020.
- CODD, E. F. ****A Relational Model of Data for Large Shared Data Banks****. Communications of the ACM, v. 13, n. 6, p. 377-387, 1970.
- ELMASRI, R.; NAVATHE, S. B. ****Sistemas de banco de dados****. 6. ed. São Paulo: Pearson, 2010.
- MONGODB, INC. ****MongoDB Server Documentation: Data Modeling Guidelines****. Disponível em: [`https://www.mongodb.com/docs/`](https://www.mongodb.com/docs/). Acesso em: 28 set. 2026.
- PANIZ, D. ****NoSQL: como armazenar os dados de uma aplicação moderna****. São Paulo: Casa do Código, 2016.